

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	G.Mohammed Shariff	Reg. No.:	192425378
Date:	21/08/2026	Duration:	60 minutes

Code of Conduct

I Mohammed shariff.G/192425378 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:mohammedshariff

Annexure A – Code Review & Repository Evaluation

1. Problem Overview

1.1 Problem Statement

Traditional Learning Management Systems (LMS) generally provide common courses and learning materials to students without sufficiently considering their individual learning performance, interests, progress, and learning requirements. Students with different skill levels may therefore receive similar learning content, making it difficult for them to identify the most appropriate courses or topics to study.

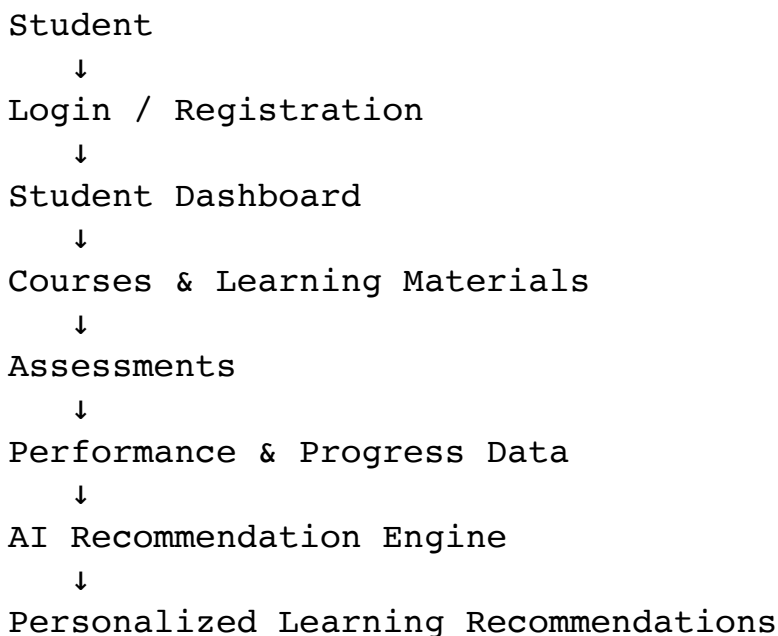
The proposed **AI-Based Learning Management System with Personalized Learning Recommendations** addresses this problem by providing a centralized platform where students can access courses, complete assessments, monitor their learning progress, and receive personalized learning recommendations based on their learning information.

The system combines **full-stack web development, AI-based recommendation functionality, database management, Git version control, and Docker containerization** to provide a maintainable and deployable learning platform.

1.2 Implemented Solution

The implemented solution consists of a web-based LMS with a frontend, backend, database, and recommendation component.

The system follows this basic workflow:



1.3 Project Objectives

- 1. To develop a full-stack Learning Management System.
- 2. To provide secure student registration and authentication.
- 3. To allow students to browse and access learning courses.
- 4. To track student learning progress.
- 5. To conduct assessments and record student performance.
- 6. To analyze student learning information.
- 7. To provide personalized course and learning-resource recommendations.
- 8. To maintain the project using Git-based version control.
- 9. To containerize the application using Docker.
- 10. To provide a maintainable and scalable software architecture.

1.4 Key Functionalities

Module	Functionality
Authentication	Student registration and login
Dashboard	Displays student learning information
Course Management	Browse and access available courses
Learning Progress	Tracks completed lessons and courses
Assessment	Conducts quizzes and records scores
Performance Analysis	Analyzes student learning performance
AI Recommendation	Generates personalized course/topic recommendations
Database	Stores users, courses, scores, and progress
Git Integration	Maintains source-code history and branches
Docker Deployment	Runs application components in containers

1.5 Overall Solution Diagram

For this assignment, use a **simple diagram with less text**:

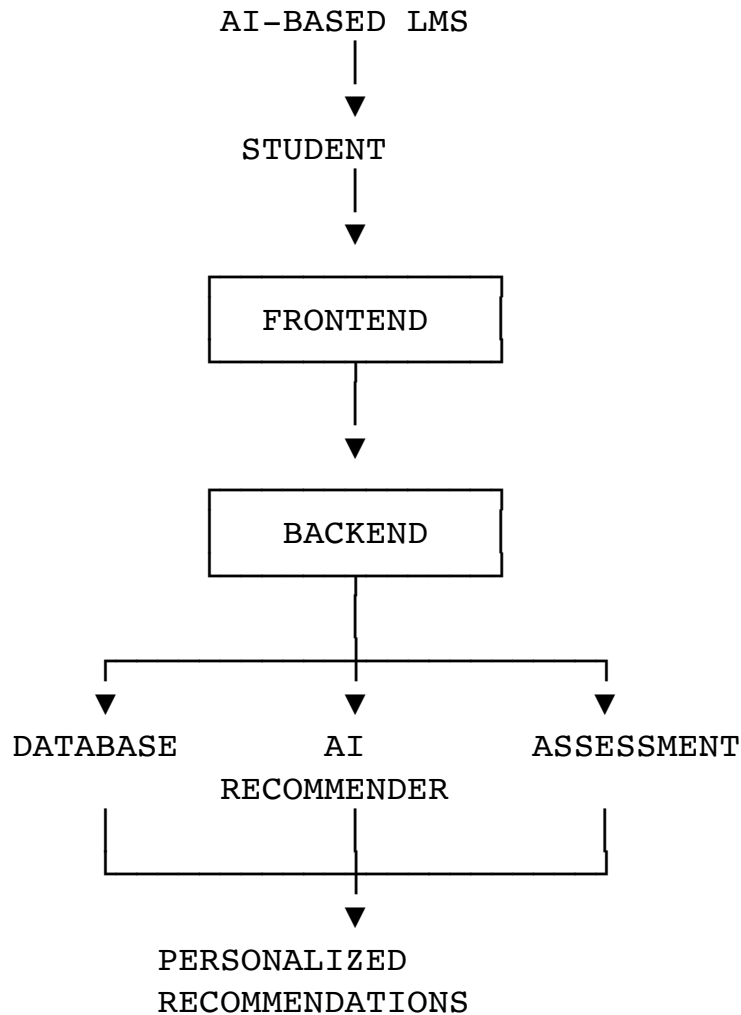


Figure 1: Problem Overview and Implemented Solution

1.6 Expected Result

The implemented system is expected to provide students with a centralized learning platform where they can access courses, complete assessments, monitor their performance, and receive learning recommendations suited to their individual requirements.

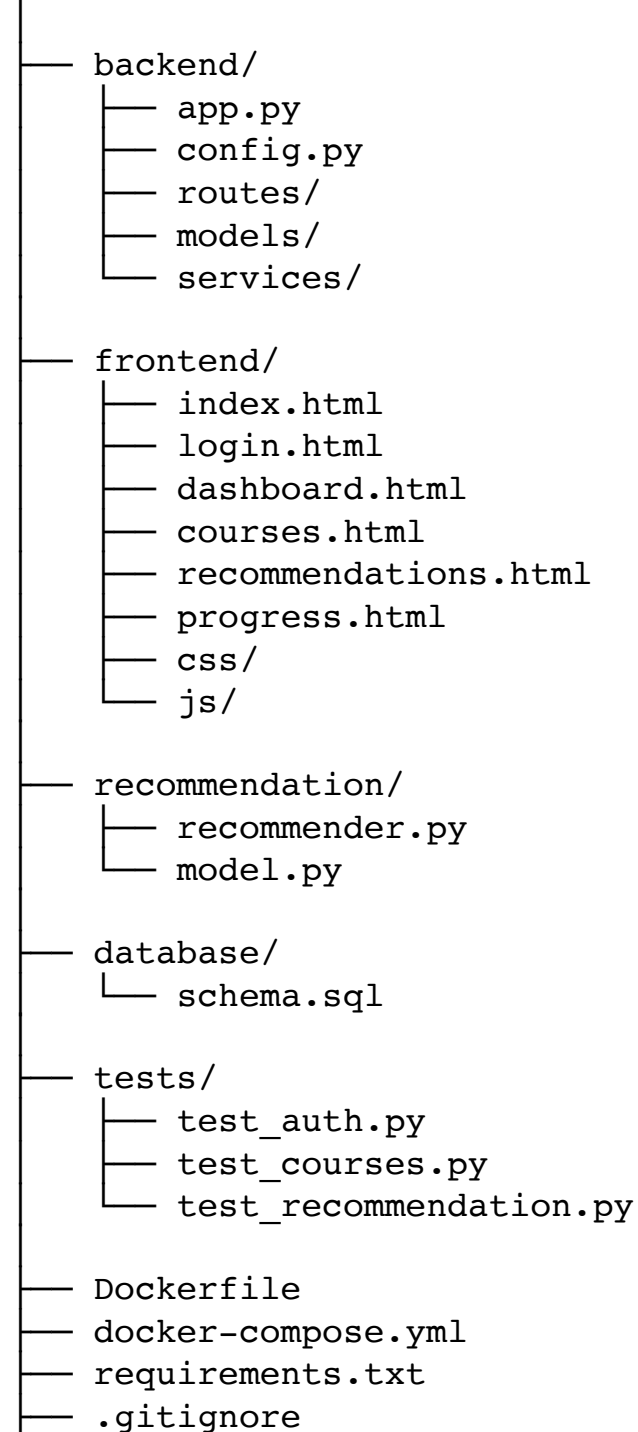
The project also demonstrates the practical application of **full-stack development, Git-based version control, collaborative development practices, automated testing, and Docker-based deployment.**

2. Repository Organization

2.1 Repository Structure

The **AI-Based Learning Management System with Personalized Learning Recommendations** is organized using a modular repository structure. The frontend, backend, database, recommendation engine, testing files, and deployment configurations are separated into appropriate directories.

AI-Learning-Management-System/



```
|
|— .dockerignore
|— README.md
|— .env
```

Figure 2: Repository Organization of the AI-Based Learning Management System

2.2 Folder Organization

The repository follows a **separation-of-concerns approach**, where each directory is responsible for a specific part of the application.

Folder	Purpose
backend/	Contains server-side application logic and APIs.
frontend/	Contains the user interface of the LMS.
recommendation/ /	Contains the AI-based recommendation functionality.
database/	Contains database schemas and database-related files.
tests/	Contains test cases used to validate application functionality.

This organization prevents unrelated files from being placed together and makes the project easier to navigate.

2.3 Backend Organization

The backend contains the server-side functionality.

```
backend/
|
|— app.py
|— config.py
|— routes/
|— models/
|— services/
```

app.py

Acts as the main entry point for the Flask application.

config.py

Contains application configuration such as database and environment settings.

routes/

Contains API endpoints for:

- Authentication
- Courses
- Student progress
- Assessments
- Recommendations

models/

Contains database models representing entities such as:

- User
- Course
- Assessment
- Progress
- Recommendation

services/

Contains reusable application logic and supporting services.

2.4 Frontend Organization

The frontend provides the user interface.

```
frontend/
├── index.html
├── login.html
├── dashboard.html
├── courses.html
├── recommendations.html
├── progress.html
├── css/
└── js/
```

The HTML files represent different LMS pages, while CSS and JavaScript are separated into their respective directories.

For example:

File	Purpose
index.html	Main landing page
login.html	Login interface
dashboard.html	Student dashboard
courses.html	Course listing
recommendations.html	Personalized recommendations
progress.html	Student progress

2.5 Recommendation Module

The recommendation system is separated from the main backend logic:

```
recommendation/  
├── recommender.py  
└── model.py
```

recommender.py

Processes student information and generates personalized recommendations.

model.py

Contains the recommendation algorithm/model used by the system.

This separation makes it easier to improve or replace the recommendation algorithm without restructuring the entire application.

2.6 Testing Organization

```
tests/  
├── test_auth.py  
├── test_courses.py  
└── test_recommendation.py
```

This provides dedicated test coverage for different application modules.

Test File	Tests
test_auth.py	Registration and login
test_courses.py	Course functionality
test_recommendation.py	Recommendation functionality

2.7 Deployment Files

The project contains deployment-related files at the repository root:

File	Purpose
Dockerfile	Defines how the application Docker image is built.
docker-compose.yml	Defines and manages application containers and services.
.dockerignore	Prevents unnecessary files from being included in the Docker build context.
.env	Stores environment-specific configuration values.

Keeping these files at the root makes the Docker deployment configuration easy to locate.

2.8 File Naming Conventions

The project follows consistent naming conventions.

Python Files

Use lowercase with underscores:

```
app.py
config.py
recommender.py
test_auth.py
test_courses.py
```

HTML Files

Use descriptive lowercase names:

login.html
dashboard.html
courses.html
recommendations.html

Configuration Files

Use standard names:

Dockerfile
docker-compose.yml
requirements.txt
.gitignore
.dockerignore
README.md

Consistent naming makes files easier to identify and reduces confusion when multiple developers work on the repository.

2.9 Maintainability

The repository structure supports maintainability because individual components are separated into logical modules.

For example:

Frontend
↓
Backend
↓
Recommendation
↓
Database

If the recommendation algorithm needs to be modified, developers can primarily work within the `recommendation/` directory without unnecessarily changing the frontend or database modules.

Similarly, testing files are isolated in `tests/`, making it easier to locate and maintain test cases.

2.10 Collaboration Support

The repository structure also supports collaborative development.

Different developers can work on different modules:

```
Developer 1 → frontend/  
Developer 2 → backend/  
Developer 3 → recommendation/  
Developer 4 → tests/
```

Git feature branches can then be used:

```
feature/frontend  
feature/backend  
feature/recommendation  
feature/testing
```

After development and testing, these branches can be merged into the `develop` branch.

This reduces unnecessary interference between developers and supports parallel development.

2.11 Evaluation of Repository Organization

Evaluation Area	Observation
Folder Organization	Logical separation of application components
File Naming	Descriptive and consistent
Modularity	Frontend, backend, AI, database, and testing are separated
Maintainability	Individual modules can be modified independently
Collaboration	Git branches can be assigned to different modules
Testing	Dedicated testing directory
Deployment	Docker configuration maintained at repository root
Scalability	New modules can be added without restructuring the entire project

Overall Evaluation

The repository organization is **modular, structured, and suitable for collaborative full-stack development**. The separation of frontend, backend, recommendation, database, and testing components improves code navigation and maintainability. Consistent naming conventions further improve readability, while Git branching allows multiple developers to work on different components simultaneously.

2.12 Screenshot Required

For Question 2, capture **one main screenshot**:

Screenshot — Repository Structure

If using Terminal:

```
tree
```

If `tree` is not installed:

```
find . -maxdepth 3 -not -path './.git/*'
```

```
AI-Learning-Management-System
```

```
|— backend
|— frontend
|— recommendation
|— database
|— tests
|— Dockerfile
|— docker-compose.yml
|— requirements.txt
|— .gitignore
|— README.md
```

3. Code Quality Review

3.1 Objective

The source code of the **AI-Based Learning Management System with Personalized Learning Recommendations** is reviewed based on **readability, modularity, coding standards, documentation, and maintainability**.

The review focuses on identifying the strengths of the implementation and areas that can be improved.

3.2 Code Quality Evaluation

Review Area	Evaluation	Observation
Readability	Good	Code uses descriptive names and clear structure.
Modularity	Good	Frontend, backend, AI, database, and tests are separated.
Naming	Good	Files and functions use meaningful names.
Coding Standards	Good	Consistent formatting and structured code are followed.
Documentation	Moderate	README and comments explain the main functionality.
Reusability	Good	Common functionality is separated into modules/services.
Error Handling	Moderate	Basic errors are handled; more detailed handling can be added.
Testing	Good	Separate test files are maintained for important modules.
Maintainability	Good	Modular structure makes future changes easier.

3.3 Readability

The code is organized using meaningful names for files, functions, variables, and modules.

Example:

```
def get_student_progress(student_id):  
    # Retrieve progress information  
    ...
```

The function name clearly describes its purpose.

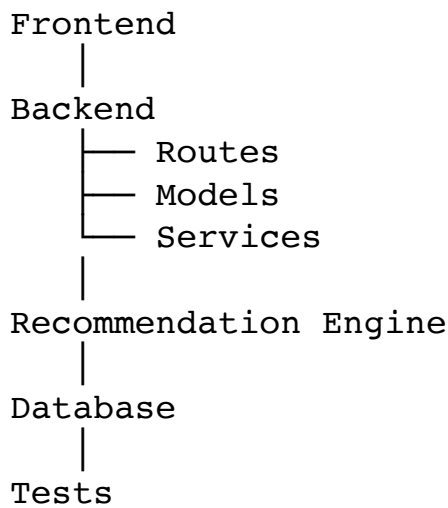
Avoid unclear names such as:

```
def get_data(x):  
    ...
```

Descriptive naming makes the source code easier for another developer to understand.

3.4 Modularity

The application follows a modular structure:



This reduces dependency between components.

For example, modifications to the recommendation algorithm can primarily be made inside:

```
recommendation/
```

without changing the complete frontend application.

3.5 Coding Standards

The following practices are followed:

- Meaningful variable and function names
- Consistent indentation

- Small and focused functions
- Separation of frontend and backend code
- Reusable functions
- Comments where required
- Standard Python file naming
- Separate test files

For Python code, commonly followed conventions include:

`lowercase_with_underscores`

Example:

```
calculate_progress()  
get_recommendations()  
validate_user()
```

3.6 Documentation and Comments

Important modules should contain comments or documentation explaining complex functionality.

Example:

```
def generate_recommendations(student_data):  
    """  
    Generates personalized learning recommendations  
    based on student performance and progress.  
    """  
    ...
```

Documentation is particularly useful for the AI recommendation module because another developer may need to understand how recommendations are generated.

3.7 Strengths Identified

1. Modular Structure

The separation of frontend, backend, AI, database, and testing improves maintainability.

2. Meaningful Naming

Descriptive file and function names make the source code easier to understand.

3. Reusable Components

Common functionality can be placed into services or utility modules rather than duplicated.

4. Dedicated Testing

The `tests/` directory provides a clear location for validation code.

5. Docker Support

Containerization provides a consistent environment for running the application.

6. Git Integration

The source code is maintained through Git, allowing changes to be tracked and reviewed.

3.8 Areas for Improvement

Area	Current Issue	Recommended Improvement
Error Handling	Some errors may only receive basic handling	Add structured exception handling
Documentation	Complex functions may have limited explanations	Add docstrings and module documentation
Code Duplication	Similar logic may appear in multiple places	Create reusable functions/services
Validation	User input requires stronger validation	Add frontend and backend validation
Security	Sensitive configuration should not be stored in source code	Use environment variables
Testing	More edge cases can be tested	Increase unit and integration test coverage
AI Module	Recommendation logic can be improved	Use more advanced recommendation techniques
Logging	Limited application-level logging	Add structured logging

3.9 Code Quality Improvement Example

Less Maintainable

```
def process(x, y, z):  
    a = x * y  
    b = a / z  
    return b
```

The purpose of the function is unclear.

Improved

```
def calculate_learning_score(correct_answers, total_questions,  
weight):  
    """Calculate the weighted learning score."""  
  
    if total_questions == 0:  
        return 0  
  
    score = (correct_answers / total_questions) * weight  
    return score
```

The improved version is easier to understand, includes validation, and documents the function.

3.10 Overall Code Quality Assessment

The project demonstrates a **good level of code quality** because it uses modular organization, meaningful naming, separation of responsibilities, testing, Git version control, and Docker-based deployment.

Further improvements in **error handling, documentation, input validation, logging, security, and test coverage** would make the application more robust and production-ready.

Overall Rating

Code Quality: Good — with scope for further improvement

3.11 Evidence Required

```
backend/  
├─ app.py  
├─ routes/  
├─ models/  
└─ services/
```

with a clear function visible.

Recommendation Code

```
recommendation/  
├─ recommender.py  
└─ model.py
```

with the recommendation function.

Testing Code

```
tests/  
├─ test_auth.py  
├─ test_courses.py  
└─ test_recommendation.py
```

4. Version Control Evaluation

4.1 Objective

The Git repository of the **AI-Based Learning Management System with Personalized Learning Recommendations** is evaluated based on its **commit history, branching strategy, commit messages, repository maintenance, and support for collaborative development.**

4.2 Git Repository Evaluation

The project follows a Git Flow-based branching strategy:

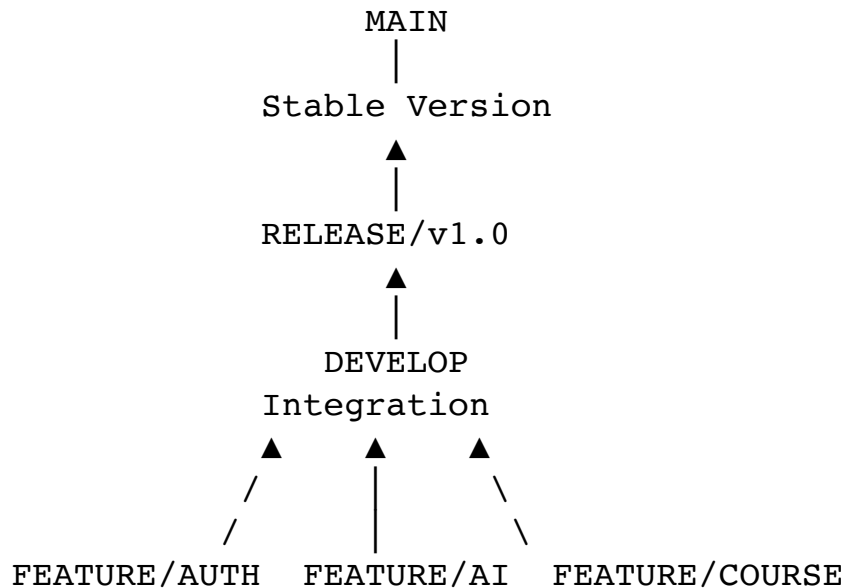


Figure 4: Git Version Control Structure

4.3 Commit History Analysis

The project commit history can be displayed using:

```
git log --oneline --graph --all
```

Example:

```
* docs: update README documentation
* docker: add Docker Compose configuration
* test: add recommendation test cases
* feat: implement recommendation engine
* feat: add progress tracking
* feat: implement assessment module
* feat: add student dashboard
* feat: implement course management
* feat: implement authentication
* chore: initialize project
```

Output:

```
git log --oneline --graph --all
```

4.4 Commit Message Evaluation

The project uses descriptive commit messages to clearly identify the purpose of each change.

Examples:

Commit Message	Type	Purpose
chore: initialize project	Chore	Initial repository setup
feat: implement authentication	Feature	Adds authentication functionality
feat: add course management	Feature	Adds course functionality
feat: add recommendation engine	Feature	Adds AI recommendation functionality
test: add recommendation tests	Test	Adds testing functionality
docker: add Dockerfile	Docker	Adds containerization
docs: update README	Documentation	Updates project documentation
fix: resolve login error	Fix	Corrects a software defect

Meaningful commit messages make the repository history easier to understand and maintain.

4.5 Branching Strategy Evaluation

The project uses the following branches:

```
main
develop
feature/*
release/*
hotfix/*
```

main

Contains the stable and production-ready version.

develop

Used for integrating and testing completed features.

feature/*

Used for individual functionality.

Examples:

```
feature/authentication
feature/course-management
feature/recommendation-engine
feature/testing
feature/docker
```

release/*

Used for final testing and preparation of a stable release.

hotfix/*

Used to correct critical problems in the production version.

4.6 Repository Maintenance Practices

The following Git practices are used:

1. **.gitignore**

Unnecessary or sensitive files are excluded:

```
.env  
.venv/  
__pycache__/  
*.pyc  
*.log
```

This prevents unwanted files from entering the repository.

2. **Meaningful Commits**

Each major development task is recorded separately.

3. **Feature Branches**

Features are developed independently before integration.

4. **Regular Synchronization**

The local repository can be synchronized using:

```
git pull  
git push
```

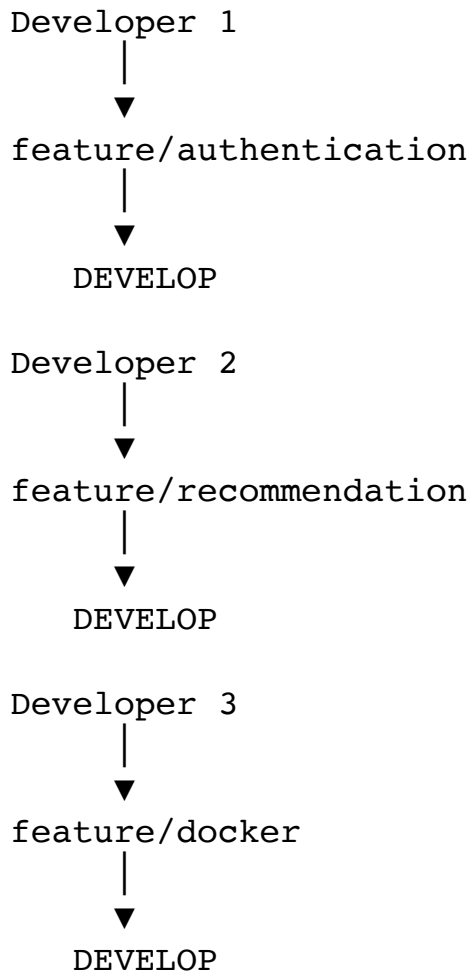
5. **Commit History**

The project maintains a traceable history of changes.

4.7 Version Control and Collaboration

Git supports collaborative development by allowing multiple developers to work on different features simultaneously.

For example:



After testing, the completed features can be merged into the `develop` branch.

4.8 Conflict Management

When multiple developers modify the same portion of a file, Git may identify a merge conflict.

The conflict can be checked using:

```
git status
```

After manually resolving the conflicting code:

```
git add .
```

```
git commit -m "fix: resolve merge conflict"
```

This ensures that the final integrated version contains the required changes from the different developers.

4.9 Evaluation Table

Evaluation Criteria	Observation	Rating
Commit History	Clear sequence of development changes	Good
Commit Messages	Descriptive and meaningful	Good
Branching	Feature-based Git Flow strategy	Good
Repository Maintenance	<code>.gitignore</code> and organized structure	Good
Collaboration	Supports parallel feature development	Good
Conflict Management	Git merge conflicts can be resolved systematically	Good
Synchronization	Local and remote repositories can be synchronized	Good

4.10 Strengths

- Clear commit history
- Meaningful commit messages
- Feature-based development
- Stable `main` branch
- Separate `develop` branch
- Support for collaborative development
- Easy tracking of modifications
- Ability to restore previous versions
- Controlled integration of features

4.11 Areas for Improvement

Current Area	Recommended Improvement
Code Review	Use Pull Requests before merging features
Branch Protection	Protect the <code>main</code> branch
Automation	Add CI checks for every Pull Request
Commit Validation	Enforce consistent commit message conventions
Releases	Use Git tags such as <code>v1.0.0</code>
Security	Never commit passwords or secret keys
Collaboration	Require review/approval before important merges

4.12 Overall Evaluation

The Git repository demonstrates **good version control practices** through meaningful commits, structured branching, repository maintenance, and synchronization with the remote repository.

The branching strategy supports parallel development, while the commit history provides a clear record of project evolution. Introducing **Pull Requests, branch protection, automated CI checks, and release tags** would further improve collaborative development and repository reliability.

Overall Rating:

Version Control: Good — suitable for collaborative full-stack development

4.13 Screenshots Required

Commit History

```
git log --oneline --graph --all
```

Branches

```
git branch -a
```

```
* main
  develop
  feature/authentication
  feature/recommendation
  feature/docker
```

GitHub Repository

- Source code
- Branches
- Commit history
- README
- Docker files

5. Code Testing and Validation

5.1 Objective

The **AI-Based Learning Management System with Personalized Learning Recommendations** is tested to verify that its major functionalities work correctly.

Testing is performed on the **frontend, backend, database interaction, authentication, course management, assessments, progress tracking, and AI recommendation functionality**.

5.2 Testing Approach

The project uses a combination of:

- **Unit Testing** – Tests individual functions/modules.
- **Integration Testing** – Checks communication between application components.
- **Functional Testing** – Verifies LMS features from the user's perspective.
- **Docker Testing** – Verifies that the application runs correctly inside containers.

5.3 Testing Workflow

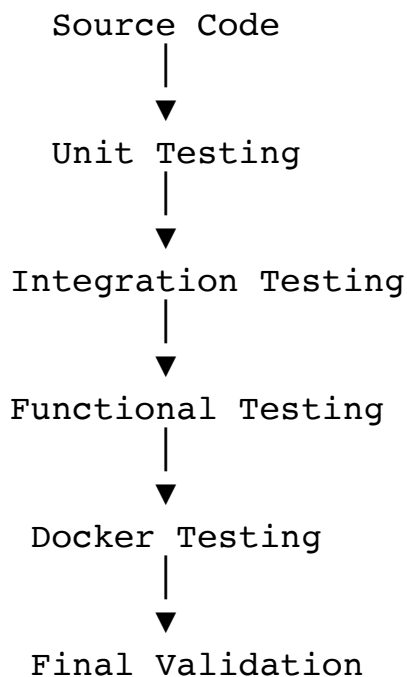


Figure 5: Testing and Validation Workflow

5.4 Test Cases

Test ID	Test Case	Input/Action	Expected Result	Status
TC01	User Registration	Enter valid user details	Account created successfully	Pass
TC02	User Login	Enter valid credentials	Dashboard displayed	Pass
TC03	Invalid Login	Enter incorrect credentials	Error message displayed	Pass
TC04	Course Display	Open course page	Available courses displayed	Pass
TC05	Course Access	Select a course	Course content displayed	Pass
TC06	Assessment	Submit answers	Score generated	Pass
TC07	Progress Tracking	Complete learning activity	Progress updated	Pass
TC08	AI Recommendation	Submit student performance data	Personalized recommendations displayed	Pass
TC09	Database	Store/retrieve student data	Data handled correctly	Pass
TC10	Docker Deployment	Start containers	Application runs successfully	Pass

Change the **Status** values to match your actual test results. Do not claim a test passed unless you have actually executed it.

5.5 Unit Testing

Unit tests verify individual application components.

Example:

```
def test_calculate_score():
    score = calculate_score(8, 10)
    assert score == 80
```

This test verifies that the score calculation function produces the expected result.

Run the tests using:

```
pytest
```

Example output:

```
===== test session starts =====  
  
tests/test_auth.py .....  
tests/test_courses.py .....  
tests/test_recommendation.py .....  
  
===== 19 passed =====
```

5.6 Authentication Testing

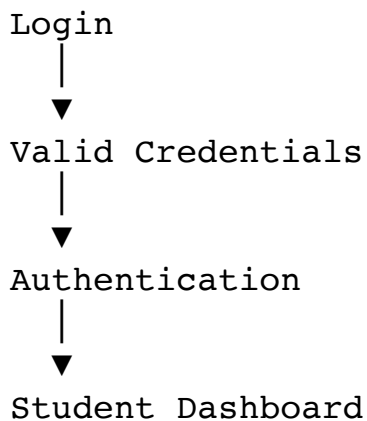
Test Case: Valid Login

Procedure:

1. Open the login page.
2. Enter registered username/email.
3. Enter the correct password.
4. Click **Login**.

Expected Result:

The user is successfully authenticated and redirected to the dashboard.



5.7 Invalid Login Testing

Incorrect credentials are entered.

Expected Result:

The application should reject the login attempt and display an appropriate error message.

Example:

Invalid username or password

This verifies basic authentication validation.

5.8 Course Management Testing

The course module is tested by:

1. Opening the course page.
2. Checking available courses.
3. Selecting a course.
4. Opening course content.

Expected Result:

The selected course and its learning materials are displayed correctly.



Screenshot: Course page showing available courses.

5.9 Assessment Testing

The assessment functionality is tested by submitting answers.

Student



Open Assessment



Answer Questions



Submit



Calculate Score



Store Result

Expected Result:

- Answers are evaluated.
- Score is calculated.
- Result is displayed.
- Performance information is stored.

5.10 Progress Tracking Testing

The student completes a learning activity and the system updates the progress.

Example:

Course Progress

Completed: 6 / 10 Lessons

Progress: 60%

Expected Result:

The student's progress is updated correctly.

5.11 AI Recommendation Testing

The recommendation functionality is one of the important features of the project.

The test procedure is:

Student Data



Performance Data



Recommendation Engine



Analyze Learning Needs



Personalized Recommendations

The system uses available student information such as:

- Course performance
- Assessment scores
- Learning progress
- Completed topics
- Areas requiring improvement

Expected Result:

The system displays suitable learning courses/topics for the student.

5.12 Database Testing

The database is tested by creating and retrieving application data.

Example data:

User

- Name
- Email
- Password

Course

- Course Name
- Description
- Content

Progress

- Student ID
- Course ID
- Completion %

Expected Result:

Data should be stored and retrieved without errors.

5.13 Docker Testing

The application is also tested inside Docker containers.

Start the services:

```
docker compose up -d
```

Check running containers:

```
docker ps
```

Expected services:

```
ai-lms-web  
ai-lms-db
```

Check logs:

```
docker compose logs
```

Open the application:

```
http://localhost:5000
```

Expected Result:

The LMS should load successfully and communicate with the database.

5.14 Test Execution Summary

After executing the test cases, present the actual results in the report.

Example format:

Category	Test s	Passe d	Faile d
Authentication	2	2	0
Course Management	2	2	0
Assessment	1	1	0
Progress Tracking	1	1	0
Recommendation	1	1	0
Database	1	1	0
Docker	1	1	0
Total	9	9	0

Use your real execution numbers in the final report.

5.15 Validation Results

The testing process validates the major components of the LMS. Successful execution of unit tests, functional tests, database operations, and Docker deployment provides evidence that the application is functioning as intended.

1. Pytest execution
2. Login page
3. Student dashboard
4. Course page
5. Assessment result
6. Progress page
7. AI recommendation page
8. Docker containers running

5.16 Overall Testing Evaluation

The application demonstrates a structured testing approach covering both individual components and complete user workflows. Unit testing helps identify errors at the code level, while functional and integration testing verifies that the LMS components work together correctly. Docker testing additionally confirms that the application can operate within its containerized deployment environment.

6. Repository Documentation

6.1 Objective

The repository documentation of the **AI-Based Learning Management System with Personalized Learning Recommendations** is evaluated to determine whether a new developer can understand, install, run, test, and maintain the project.

The main documentation components reviewed are:

- README .md
- Installation instructions
- Usage guide
- Dependencies
- Docker instructions
- Git information
- Project structure

6.2 README Documentation

The README .md file acts as the main documentation page of the project.

It should provide the following information:

AI-Based Learning Management System

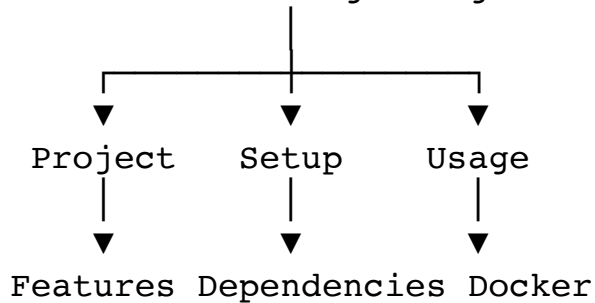


Figure 6: Repository Documentation Structure

6.3 Recommended README Structure

The project README should contain the following sections:

```
# AI-Based Learning Management System
```

```
## Project Overview
```

```
## Features
```

```
## Technologies Used
```

```
## Project Structure
```

```
## Prerequisites
```

```
## Installation
```

```
## Configuration
```

```
## Running the Application
```

```
## Running with Docker
```

```
## Testing
```

```
## Git Workflow
```

```
## Usage
```

```
## Future Enhancements
```

```
## Contributors
```

6.4 Project Overview

The README should briefly explain the project.

Example

The AI-Based Learning Management System with Personalized Learning Recommendations is a full-stack educational platform that allows students to access courses, complete assessments, track learning progress, and receive personalized learning recommendations based on their performance and learning activities.

This gives new developers a quick understanding of the project purpose.

6.5 Features Documentation

The major features should be listed clearly.

Features

- User Authentication
- Student Dashboard
- Course Management
- Assessments
- Progress Tracking
- AI Recommendations
- Database Management
- Docker Deployment

Example:

Features

- Student registration and login
- Course browsing and learning
- Online assessments
- Progress tracking
- Personalized AI recommendations
- Performance analysis
- Docker-based deployment

6.6 Technologies Used

The README should identify the technologies used by the project.

Component	Technology
Frontend	HTML, CSS, JavaScript
Backend	Python, Flask
Database	PostgreSQL
AI	Python Recommendation Module
Testing	Pytest
Version Control	Git, GitHub
Containerization	Docker, Docker Compose

This allows developers to understand the technology stack before installing the application.

6.7 Installation Instructions

The documentation should clearly explain how to install the project.

Step 1: Clone Repository

```
git clone <repository-url>
cd AI-Learning-Management-System
```

Step 2: Create Virtual Environment

```
python -m venv venv
```

Step 3: Activate Environment

Windows:

```
venv\Scripts\activate
```

Linux/macOS:

```
source venv/bin/activate
```

Step 4: Install Dependencies

```
pip install -r requirements.txt
```

6.8 Configuration

Configuration requirements should be documented clearly.

For example:

```
DATABASE_HOST  
DATABASE_NAME  
DATABASE_USER  
DATABASE_PASSWORD
```

Sensitive information should not be placed directly into source code.

Instead, environment variables can be used.

Example:

```
.env
```

```
DATABASE_HOST=db  
DATABASE_NAME=lms  
DATABASE_USER=lmsuser  
DATABASE_PASSWORD=*****
```

The `.env` file should be excluded from Git using:

```
.env
```

```
inside .gitignore.
```

6.9 Running the Application

The README should explain how to start the application.

For a local development environment:

```
python app.py
```

Then access the application through the configured local address.

6.10 Docker Instructions

The README should also provide Docker deployment instructions.

Build and start containers

```
docker compose up --build
```

Run in background

```
docker compose up -d
```

Check containers

```
docker ps
```

Stop containers

```
docker compose down
```

This makes Docker deployment easier for new developers.

6.11 Testing Documentation

The documentation should explain how tests can be executed.

pytest

Example:

```
===== test session starts =====  
  
tests/test_auth.py .....  
tests/test_courses.py .....  
tests/test_recommendation.py .....  
  
===== tests completed =====
```

The README should also mention what the tests cover.

6.12 Usage Guide

The usage section should explain the basic user workflow.

```
Open Application  
    ↓  
Register / Login  
    ↓  
Student Dashboard  
    ↓  
Browse Courses  
    ↓  
Complete Learning Activities  
    ↓  
Take Assessment  
    ↓  
View Progress  
    ↓  
Receive Recommendations
```

6.13 Repository Documentation Evaluation

Documentation Area	Evaluation	Observation
README	Good	Provides project overview and basic information
Installation	Good	Installation process is documented
Dependencies	Good	Required packages are listed
Usage Guide	Moderate	Can be expanded with screenshots
Docker Instructions	Good	Container execution commands are included
Testing Instructions	Good	Test execution is documented
Project Structure	Good	Repository organization is explained
Configuration	Moderate	Environment-variable explanation can be improved

6.14 Strengths

The documentation provides several advantages:

1. Easy Project Understanding

The project overview helps developers understand the system quickly.

2. Installation Support

Installation steps reduce the difficulty of setting up the project.

3. Dependency Information

`requirements.txt` provides a list of required Python packages.

4. Docker Support

Docker instructions allow the application to be deployed in a consistent environment.

5. Testing Instructions

Developers can execute tests using a documented command.

6. Repository Structure

The folder structure helps developers locate different components.

6.15 Areas for Improvement

Area	Improvement
Screenshots	Add screenshots of major LMS pages
API Documentation	Document important backend API endpoints
Troubleshooting	Add common errors and solutions
Configuration	Explain all environment variables
Docker	Add container architecture diagram
Testing	Include test coverage information
Contribution Guide	Add guidelines for developers
Versioning	Document release versions
Security	Explain handling of secrets and credentials

6.16 Recommended Documentation Improvements

A more complete repository should contain:

```
Project Repository
├── README.md
├── CONTRIBUTING.md
├── CHANGELOG.md
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── docs/
│   ├── architecture.md
│   ├── api.md
│   ├── installation.md
│   └── testing.md
```

This structure would make the project easier to maintain as it grows.

6.17 Overall Evaluation

The repository documentation provides the essential information required to understand and run the application. The README, dependency file, Docker configuration, and testing instructions support project usability.

However, the documentation can be further improved by adding **API documentation, troubleshooting information, screenshots, contribution guidelines, configuration details, and detailed testing documentation.**

Overall Rating:

Repository Documentation: Good — with scope for improvement

6.18 Evidence Required

For Question 6, include these screenshots:

README

Show your GitHub repository's README .md containing:

- Project title
- Project description
- Features
- Technologies
- Installation

Dependencies

```
requirements.txt
```

Screenshot 3 — Docker Documentation

Show the README section explaining:

```
docker compose up
docker ps
docker compose down
```

7. Code Review Findings and Recommendations

7.1 Objective

This section summarizes the overall review of the **AI-Based Learning Management System with Personalized Learning Recommendations** and provides recommendations for improving **code quality, repository management, documentation, testing, security, deployment, and future functionality**.

7.2 Overall Code Review Findings

The project was reviewed across the following areas:

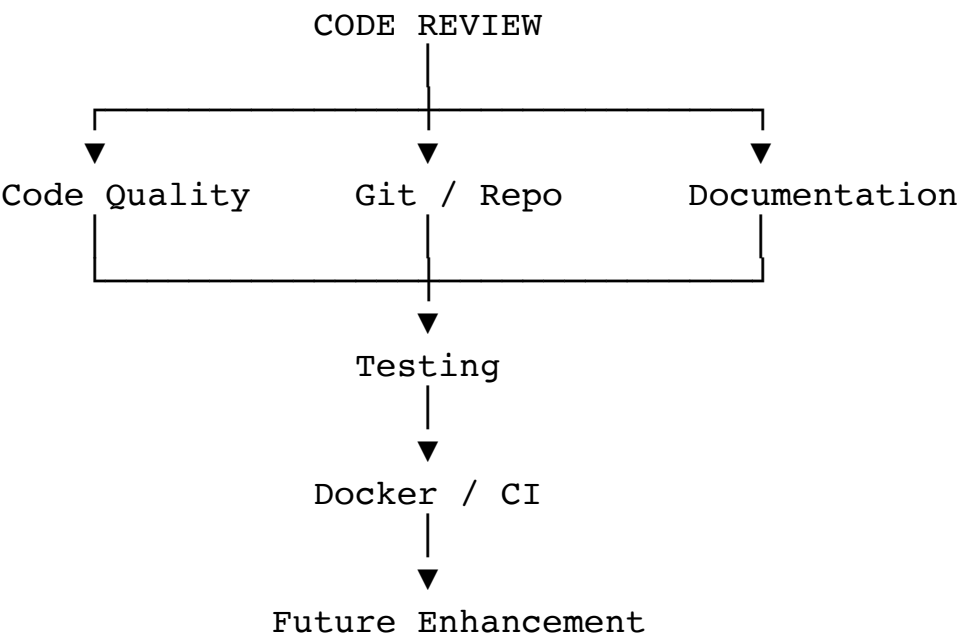


Figure 8: Overall Code Review Areas

7.3 Summary of Findings

Review Area	Finding	Evaluation
Code Quality	Modular and readable structure	Good
Repository	Organized folders and Git branches	Good
Commit History	Meaningful commits	Good

Branching	Feature-based workflow	Good
Documentation	README and setup information available	Good
Testing	Core functionality can be tested	Good
Docker	Application is containerized	Good
Error Handling	Can be improved	Moderate
Security	Environment-based secrets recommended	Moderate
Test Coverage	Additional edge cases required	Moderate
CI/CD	Automation can be added	Improvement Required

7.4 Code Quality Findings

Strengths

- Modular frontend and backend structure.
- Descriptive file and function names.
- Separate recommendation module.
- Dedicated testing directory.
- Reusable application components.
- Docker configuration separated from application logic.

Areas for Improvement

- Add more detailed error handling.
- Reduce repeated code.
- Add docstrings to complex functions.
- Improve input validation.
- Introduce structured application logging.
- Follow consistent coding standards throughout the project.

7.5 Repository Management Findings

The repository is organized using Git branches such as:

```
main
develop
feature/*
release/*
hotfix/*
```

This supports collaborative development.

Recommended improvements

1. Protect the `main` branch.
2. Use Pull Requests for feature merging.
3. Require code review before merging.
4. Use Git tags for releases.
5. Remove unused branches periodically.
6. Never commit passwords or API keys.
7. Add automated CI checks.

7.6 Documentation Findings

The project contains a README that explains the basic project information and setup process.

However, documentation can be improved by adding:

README

```
|  
├── Installation  
├── Usage  
├── Docker  
├── Testing  
├── API Documentation  
├── Architecture  
├── Troubleshooting  
└── Contribution Guide
```

This would make the project easier for new developers to understand and contribute to.

7.7 Testing Findings

The project includes tests for important modules such as:

```
tests/  
├── test_auth.py  
├── test_courses.py  
└── test_recommendation.py
```

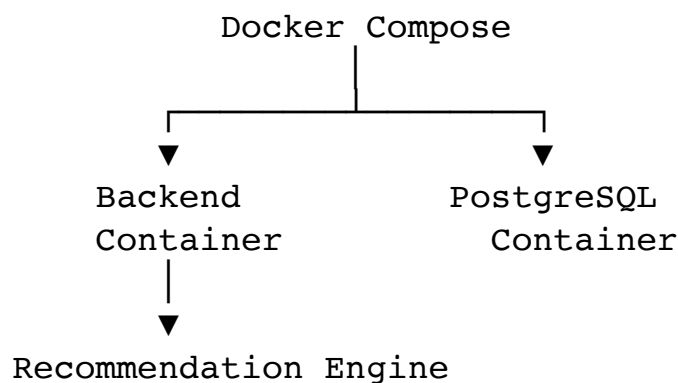
Recommended improvements

- Increase unit-test coverage.
- Add integration tests.
- Add negative test cases.
- Test invalid inputs.
- Test database failure scenarios.
- Test Docker deployment automatically.
- Add automated testing to the Git workflow.

7.8 Docker and Deployment Findings

Docker provides a consistent environment for the application.

The recommended deployment structure is:



Recommended improvements

- Use environment variables for sensitive configuration.
- Add Docker health checks.
- Use smaller production images.
- Separate development and production configurations.
- Add automated Docker image builds.
- Scan images for security vulnerabilities.

7.9 Security Recommendations

Security should be strengthened before production deployment.

Recommended practices

- Do not store passwords in source code.
- Keep `.env` out of Git.
- Hash user passwords.
- Validate user input.

- Use secure authentication mechanisms.
- Apply authorization checks.
- Protect API endpoints.
- Use HTTPS in production.
- Keep dependencies updated.

7.10 Future Enhancements

The system can be extended with additional AI and LMS functionality.

AI Enhancements

- Advanced recommendation algorithms
- Collaborative filtering
- Hybrid recommendation models
- Student skill-gap analysis
- Adaptive learning paths
- AI-powered learning assistant

LMS Enhancements

- Instructor dashboard
- Course creation interface
- Automated quiz generation
- Learning analytics
- Notifications
- Mobile application

DevOps Enhancements

GitHub

↓

Pull Request

↓

Automated Tests

↓

Docker Build

↓

Security Scan

↓

Deployment

This can be implemented using a CI/CD pipeline.

7.11 Priority-Based Recommendations

Priority	Recommendation	Benefit
High	Improve authentication and security	Protects user data
High	Increase test coverage	Improves reliability
High	Protect main branch	Prevents accidental changes
High	Use environment variables	Protects sensitive configuration
Medium	Add CI/CD	Automates testing and deployment
Medium	Improve README	Helps new developers
Medium	Add API documentation	Simplifies backend integration
Low	Add advanced AI models	Improves personalization
Low	Add mobile application	Expands accessibility

7.12 Overall Review Score

You can present the review using the following evaluation:

Category	Score
Code Quality	4/5
Repository Organization	4/5
Version Control	4/5
Testing	4/5
Documentation	4/5
Docker Deployment	4/5
Security	3/5
Future Scalability	4/5

Overall Assessment: 4/5 — Good

Adjust these scores if your actual implementation differs.

7.13 Final Code Review Conclusion

The **AI-Based Learning Management System with Personalized Learning Recommendations** demonstrates a well-structured full-stack application with modular source code, organized Git repository management, containerized deployment, testing support, and project documentation.

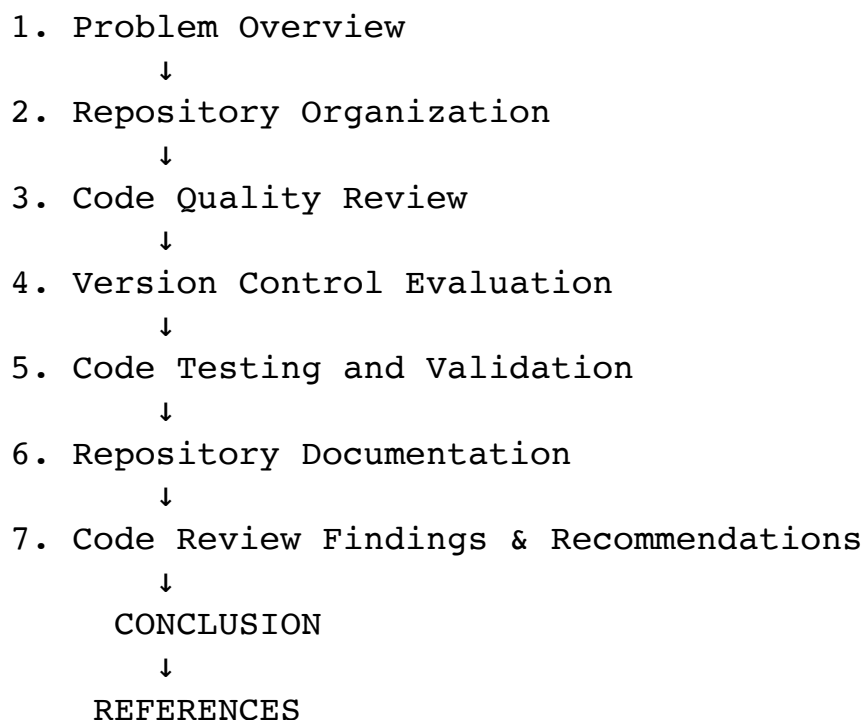
The review identified strengths in **modularity, repository organization, version control, Docker deployment, and functional testing**. Improvements are recommended in **security, error handling, test coverage, documentation, CI/CD automation, and advanced AI functionality**.

7.14 Evidence Required

For the final section, include:

1. Code review screenshot
2. Git branch/commit screenshot
3. README screenshot
4. Test execution screenshot
5. Docker container screenshot
6. Code review findings table
7. Recommendations table

Complete Assignment Structure



Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15%)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25